Prompt 01 Usage Guide

Using GitHub Copilot in Visual Studio Code to Generate a Living PRD

CISC 593/594 - Product Requirements Document

Purpose
Use the attached Prompt-01-Generate-PRD.md file with GitHub Copilot in Visual Studio Code to create or update a living Product Requirements Document based on the actual contents of your repository.

Required Output

The completed PRD must be saved in your repository at:

```
/docs/Product_Requirements_Document.md
```

1. Open the Complete Repository in Visual Studio Code

- Clone the repository or pull the latest changes.
- Open Visual Studio Code.
- Select File > Open Folder.
- Open the repository root folder, not only a source-code subfolder.

Copilot needs the complete workspace so it can review source code, documentation, tests, configuration files, dependency files, and CI/CD workflows.

2. Confirm GitHub Copilot Is Available

- Sign in to GitHub in Visual Studio Code.
- Open Copilot Chat.
- Use Agent mode so Copilot can inspect files and create or update the PRD.

3. Add the Prompt File to the Repository

Download the attached prompt from Canvas and create this folder if it does not exist:

```
.github/prompts/
```

Copy the prompt into that folder and rename it:

```
.github/prompts/generate-prd.prompt.md
```

The repository should resemble:

```
ProjectName/
├── .github/
│   └── prompts/
│       └── generate-prd.prompt.md
├── docs/
├── src/ or other source folders
├── tests/
└── README.md
```

4. Run the Prompt

Open Copilot Chat and enter:

```
/generate-prd
```

Then add this instruction:

```
Analyze the complete repository using #codebase. Create or update
/docs/Product_Requirements_Document.md. Base the document on evidence
from the repository. Do not invent features, requirements, test results,
risks, tools, or implementation details. Mark unsupported information
as To Be Completed.
```

The full request should resemble:

```
/generate-prd

Analyze the complete repository using #codebase. Create or update
/docs/Product_Requirements_Document.md. Base the document on evidence
from the repository. Do not invent features, requirements, test results,
risks, tools, or implementation details. Mark unsupported information
as To Be Completed.
```

5. Provide Important Repository Context

Copilot should inspect the complete repository. Attach or reference important files and folders when needed.

```
#README.md
#docs
#src
#tests
#requirements.txt
#package.json
```

Make sure Copilot reviews:

- README and existing project documentation
- source-code folders

- current and planned features
- existing requirements or proposals
- test files
- dependency and configuration files
- CI/CD workflow files
- existing risk or architecture documents

6. Review the Generated PRD

Before accepting changes, verify all of the following:

- Level-1 capabilities represent the major software capabilities.
- The number of Level-1 capabilities follows Miller's Law: approximately 7 plus or minus 2.
- Every capability begins with an action verb.
- Level-2 capabilities are hierarchically numbered beneath the correct Level-1 capability.
- Undesirable events trace to Level-2 capabilities.
- Risks trace to the corresponding undesirable events.
- Likelihood and impact use the required 1-5 scales.
- Each risk score equals Likelihood x Impact.
- Risk prioritization is ordered from highest to lowest score.
- Mitigations are classified as Pure Software, Hybrid, or Pure Hardware.
- Functional requirements use the required ABC structure.
- Requirements trace back to Level-2 capabilities.
- Quality and performance requirements are measurable.
- Unsupported assumptions are marked To Be Completed.

You are responsible for verifying the accuracy of all AI-generated content. Do not automatically accept every suggestion.

7. Correct Unsupported or Incorrect Content

Continue the Copilot conversation when corrections are needed. Examples:

> Capability 3 is written as a component name rather than an action-oriented capability. Rewrite it using an action verb without changing the intended scope.
>
> Review the source code again and identify which Level-2 capabilities are actually implemented. Mark the remaining capabilities as planned.
>
> Several requirements do not follow the ABC format. Rewrite them using: Actor shall process/manage/perform behavior within constraint.
>
> Do not invent performance thresholds. Mark any unsupported threshold as To Be Completed.

8. Save and Verify the PRD

Confirm that the completed file is located at:

```
/docs/Product_Requirements_Document.md
```

Check that the Markdown renders correctly in:

- Visual Studio Code Markdown Preview
- GitHub

9. Commit and Push the Changes

Commit both the reusable prompt and the generated PRD.

Recommended commit message:

```
docs: establish living product requirements document
```

The repository should contain:

```
.github/prompts/generate-prd.prompt.md
docs/Product_Requirements_Document.md
```

10. Maintain the PRD as a Living Document

Update the PRD whenever:

- a capability is added, changed, or removed
- a new undesirable event or risk is identified
- a risk score changes
- a mitigation is implemented or revised
- a functional requirement changes
- a quality or performance requirement changes
- the software moves to a new version
- instructor feedback results in corrections

Each significant update must also update the PRD revision-history table and be committed to GitHub with a meaningful commit message.

Alternative Method

If Visual Studio Code does not recognize the prompt as a slash command:

1. Open Prompt-01-Generate-PRD.md.

2. Copy the complete prompt text.

3. Paste it into Copilot Chat.

4. Add the instruction shown below.

5. Run the request in Agent mode.

6. Review all proposed changes before accepting them.

```
Use #codebase to review this repository. Create or update
/docs/Product_Requirements_Document.md.
```

Submission Reminder

Required file	/docs/Product_Requirements_Document.md
Prompt location	/.github/prompts/generate-prd.prompt.md
Submission	Direct GitHub link to the completed PRD
Identification	Branch name and commit SHA being evaluated